Лекция 15. Интеграция и проверяемый выпуск ML-системы

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанные работы: ЛР15 «Нагрузочный протокол», ЛР16 «Интеграция и защита проекта»

1. Цель лекции

Научить студентов собирать результаты всего семестра в проверяемый выпуск: связать identities кода, данных, модели, API и контейнера, подтвердить gates, оформить исключения, независимое approval и воспроизводимый rollback. Итогом является не презентация, а доказуемое состояние системы.

2. Результаты обучения

Студент сможет:

- определить release identity и границы выпуска;
- построить manifest со ссылками и hashes evidence;
- отличить alias модели от точной model version;
- применить build once, promote the same artifact;
- разделить подготовку и одобрение выпуска;
- оформить временное исключение без скрытого обхода gate;
- проверить происхождение и целостность артефакта;
- выполнить и доказать rollback на точные предыдущие identities.

3. Интеграция не равна повтору ЛР

На финальной неделе команда не пересказывает 15 отчётов. Она доказывает, что артефакты образуют одну цепочку: commit создал pipeline run, run зарегистрировал exact model version, сервис использует эту версию и image, monitoring относится к тому же выпуску, а rollback возвращает известное состояние.

4. Release identity

Release identity включает version/tag, full source commit SHA, data/feature contract versions, MLflow run и exact model version, immutable image ref, OpenAPI version и environment. Если хотя бы один элемент обозначен «latest» или «текущий», состояние нельзя воспроизвести.

5. Release manifest

Manifest — машинно-читаемый индекс выпуска. Он не копирует все отчёты, а связывает identities, gates, SHA-256 evidence, approvals, ограничения и rollback. Manifest тоже является артефактом и фиксируется release tag или hash.

6. Evidence graph

Удобно читать manifest как граф: source commit → CI run → image; data contract → prepared dataset → MLflow run → model version; model/API/image → monitoring; risk/capacity gates → release decision. Несвязанная карточка или dashboard не является доказательством именно этого выпуска.

7. Код

Используется полный 40-символьный commit SHA. Ветка и PR важны для процесса review, но могут перемещаться. Release tag удобен для человека, однако manifest хранит и tag, и commit, чтобы обнаружить перемещение или ошибочную сборку.

8. Данные и признаки

Фиксируются data contract version, identity исходного/подготовленного набора, временные границы и feature contract. Hash без provenance подтверждает байты, но не право и смысл данных; data card и license evidence остаются частью выпуска.

9. MLflow run и model version

Run ID связывает параметры, метрики и артефакты обучения. Registry model version неизменяема как конкретный объект; alias `champion` — управляемый указатель. Manifest хранит оба и resolved URI `models:/name/version`, чтобы последующее перемещение alias не переписало историю выпуска.

10. Container identity

Плавающий `latest` запрещён. Надёжный вариант — registry digest; учебный минимум — tag, равный full commit SHA при включённой политике неизменяемости. SemVer удобен пользователю, но без immutability может быть перезаписан и потому дополняется digest или commit identity.

11. API identity

Версия OpenAPI и contract tests должны соответствовать запущенному image. Совместимость описывается явно: backward-compatible или breaking. Изменение схемы без version/review является release risk, даже если модель не изменилась.

12. Build once, promote the same artifact

Нельзя заново собирать image для production после теста candidate: новая сборка может получить иные зависимости. Один artifact проходит gates и продвигается между средами. Конфигурация среды отделена от содержимого image и также контролируется.

13. Release gates

Gate имеет имя, policy version, вход, статус и evidence. В курсе обязательны data contract, point-in-time features, monitoring decision, responsible AI, capacity, model card и security. Команда может добавить, но не молча удалить обязательный gate.

14. Gate pass и доказательство

Статус `pass` без пути, CI run или hash не проверяем. Manifest builder читает файл и вычисляет SHA-256. Если evidence изменилось после решения, `verify` завершается ошибкой. Это защищает от «тихого исправления» отчёта после approval.

15. Управляемое исключение

Не каждый alert означает немедленный stop. Например, drift при стабильном отложенном качестве может быть принят на короткий срок. Статус `accepted_with_investigation` требует причину, владельца, срок пересмотра и mitigation. Бессрочное исключение превращается в скрытое отключение gate.

16. Draft и ready

Учебный example остаётся `draft`, потому что не имеет реального CI и независимого approval. Статус `ready` возникает только при успешном CI и разных prepared/approved identities. Наличие исключения даёт `ready_with_exception`, а не маскируется как обычный pass.

17. Separation of duties

Автор изменения не должен единолично одобрять выпуск. Для маленькой команды reviewer может быть другим участником, преподавателем или назначенным risk owner. Важно не название роли, а независимая проверка критерия и возможность остановить release.

18. CI и CD

CI доказывает качество конкретного commit: lint, tests, contracts, model/data/security gates. CD продвигает уже проверенный artifact с environment protection и approval. Зелёный CI не означает, что production deployment автоматически разрешён.

19. GitHub environment protection

Environment может требовать reviewer, ограничивать ветки и выдавать secrets только deployment job. В учебном проекте фактический внешний deploy не обязателен, но students должны показать, где была бы граница approval и почему секрет не нужен pull-request job.

20. Artifact provenance и attestation

Attestation криптографически связывает artifact с workflow, repository, commit и событием. Она усиливает provenance, но не доказывает безопасность, качество модели или корректность policy. Польза возникает только при проверке attestation потребителем.

21. Минимальные permissions

Workflow объявляет минимальные `permissions`. Право `contents: read` достаточно большинству quality jobs; публикация package, OIDC или attestation получает write только в нужном job. Full-SHA pinning или lockfile для actions снижает риск подмены зависимости workflow.

22. Rollback как отдельный release

Rollback — управляемое изменение состояния с причиной, approver и evidence. Он возвращает exact previous model version и immutable image, затем выполняет smoke/readiness checks. «Поставить старое» без identities не является процедурой.

23. Model и application rollback

Иногда достаточно переместить alias `champion`, иногда требуется вернуть image/API. Если новая версия API несовместима со старой моделью, частичный rollback опасен. Manifest фиксирует совместимую пару model/image и порядок действий.

24. Roll forward

Небольшая безопасная ошибка может исправляться новым release быстрее rollback. Выбор зависит от времени, риска несовместимости, данных и доступной предыдущей версии. Решение и критерий записываются, а не принимаются только по интуиции во время инцидента.

25. Post-release verification

После deploy/rollback проверяются resolved identities, readiness, smoke prediction, error rate и ключевые SLO. Успех команды deployment ещё не означает, что сервис использует нужную модель и отвечает корректно.

26. Audit trail

Issue формулирует критерий; PR содержит diff и review; CI — результаты; manifest — identities; approval — решение; monitoring — последствия; rollback/postmortem — реакцию. Удаление первой неуспешной попытки разрушает обучение и аудит.

27. Сценарий итоговой демонстрации

За 10–12 минут команда показывает manifest, воспроизводимость данных/признаков, exact model, контейнер/API, observability, governance/capacity decision и rollback. Каждая операция имеет заранее определённый ожидаемый результат и запасной безопасный путь.

28. Индивидуальная защита

Общая демонстрация не подтверждает понимание каждого. Студент за 3–4 минуты показывает личный артефакт, прослеживает его связи и выполняет изменение/диагностику: проверить hash, resolved alias, сломать gate в копии, объяснить alert или провести smoke после rollback.

29. Оценка вклада

Количество commit и объём речи не являются метриками вклада. Принимаются код, тест, contract, manifest evidence, содержательный review, runbook, security/capacity decision и диагностика, если результат встроен в общую версию и воспроизводим.

30. Типичные анти-паттерны

- tag без commit SHA;
- alias без resolved version;
- `latest` или пересобранный production image;
- pass без evidence/hash;
- исключение без owner/expiry;
- author равен approver;
- rollback только на словах;
- dashboard из другого периода;
- общая презентация вместо индивидуальной защиты.

31. Демонстрация преподавателя

1. Собрать draft manifest из example spec.
2. Проверить hashes командой `release verify`.
3. Изменить копию evidence и показать отказ.
4. Заменить image на `latest` и показать отказ.
5. Добавить CI success/independent approval и получить `ready_with_exception`.
6. Проследить exception drift до owner и срока.
7. Сопоставить previous model/image с rollback steps.

32. Вопросы аудитории

1. Почему alias недостаточен для исторического manifest?
2. Чем digest отличается от SemVer tag?
3. Почему attestation не доказывает безопасность?

4. Когда допустим `ready_with_exception`?
5. Зачем хешировать evidence?
6. Почему partial rollback может быть опасен?
7. Что проверяется после deployment?
8. Как отличить личный вклад от общей демонстрации?

33. Связь с ЛР16 и Лекцией 16

ЛР16 реализует полный release review и защиту. Лекция 16 завершит курс архитектурным разбором, типичными отказами, ретроспективой инженерных решений и подготовкой к экзаменационному контролю.

34. Официальные источники

- GitHub artifact attestations: https://docs.github.com/en/actions/concepts/security/artifact-attestations
- GitHub deployment environments: https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments
- GitHub Actions security: https://docs.github.com/en/actions/how-tos/secure-your-work
- MLflow Model Registry workflow: https://mlflow.org/docs/latest/ml/model-registry/workflow/